

JAVA-OOP CONCEPTS

Class & Object

Summary: A class is a blueprint or template used to create objects. It contains variables (data members) and methods (functions).

An object is an instance of a class that represents real-world entities. Objects are used to access the properties and methods of a class.

Program :

```
class Student {  
  
    String name = "Girishma";  
    int age = 22;  
  
    void display() {  
        System.out.println("Name: " + name);  
        System.out.println("Age: " + age);  
    }  
  
    public static void main(String[] args) {  
  
        Student s = new Student(); // Object creation  
        s.display();  
  
    }  
}
```

Output:

Name: Girishma

Age: 22

```
C:\Users\SLA 1035 Girishma K\Desktop>javac Students.java  
C:\Users\SLA 1035 Girishma K\Desktop>java Students.java  
Name: Girishma  
Age: 22
```

Main Concepts of OOP:

- 1.Encapsulation
- 2.Inheritance
- 3.Ploymorphism
- 4.Abstraction

1. Encapsulation:

Encapsulation means wrapping data and methods into a single unit called a class. It helps to protect data.

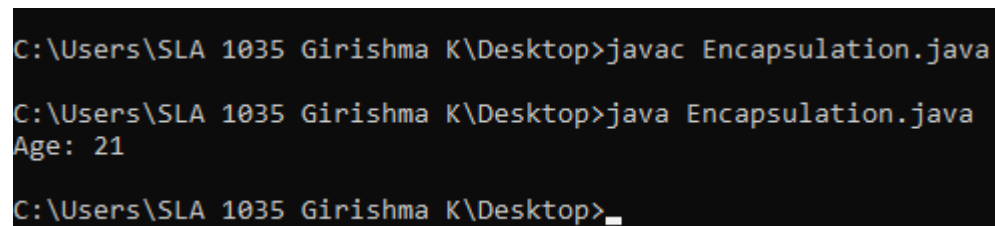
Program:

```
public class Encapsulation {  
  
    private int age = 21;  
  
    public void display() {  
        System.out.println("Age: " + age);  
    }  
  
    public static void main(String[] args) {
```

```
Encapsulation obj = new Encapsulation();  
obj.display();  
}  
}
```

Output:

Age:21



```
C:\Users\SLA 1035 Girishma K\Desktop>javac Encapsulation.java  
C:\Users\SLA 1035 Girishma K\Desktop>java Encapsulation.java  
Age: 21  
C:\Users\SLA 1035 Girishma K\Desktop>_
```

2. Inheritance:

Inheritance allows one class to acquire the properties and methods of another class. It helps in code reusability.

Program:

```
public class Inheritance {  
    public static void main(String[] args) {  
        Dog d = new Dog();  
        d.sound();  
        d.bark();  
    }  
}
```

```
class Animal {  
    void sound() {
```

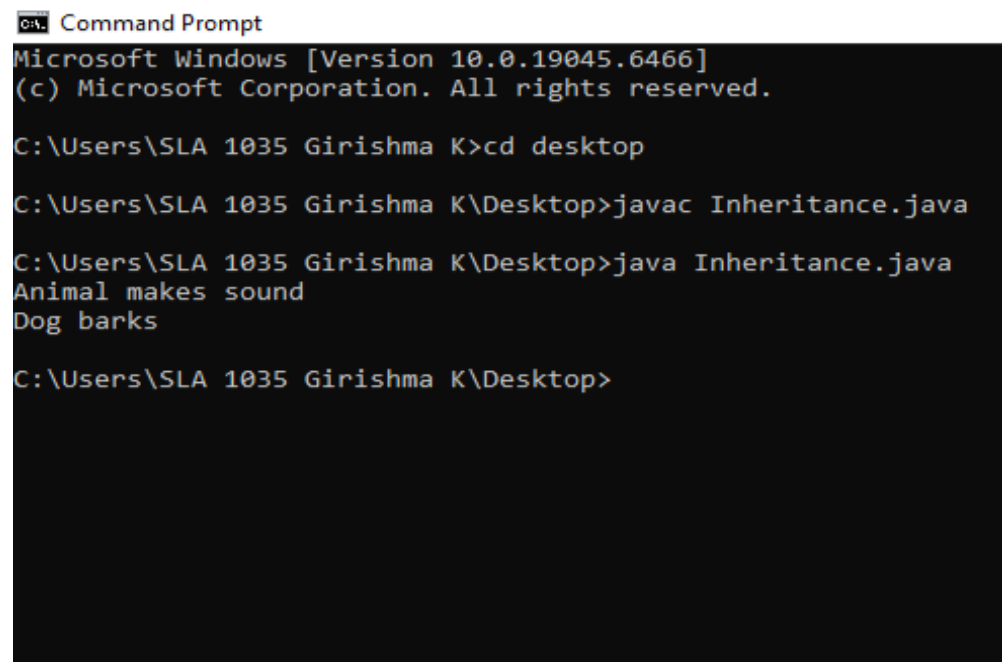
```
        System.out.println("Animal makes sound");
    }
}
```

```
class Dog extends Animal {
    void bark() {
        System.out.println("Dog barks");
    }
}
```

Output:

Animal makes sound

Dog barks

A screenshot of a Windows Command Prompt window. The title bar reads "C:\> Command Prompt". The window content shows the following text: "Microsoft Windows [Version 10.0.19045.6466] (c) Microsoft Corporation. All rights reserved. C:\Users\SLA 1035 Girishma K>cd desktop C:\Users\SLA 1035 Girishma K\Desktop>javac Inheritance.java C:\Users\SLA 1035 Girishma K\Desktop>java Inheritance.java Animal makes sound Dog barks C:\Users\SLA 1035 Girishma K\Desktop>".

```
C:\> Command Prompt
Microsoft Windows [Version 10.0.19045.6466]
(c) Microsoft Corporation. All rights reserved.

C:\Users\SLA 1035 Girishma K>cd desktop

C:\Users\SLA 1035 Girishma K\Desktop>javac Inheritance.java

C:\Users\SLA 1035 Girishma K\Desktop>java Inheritance.java
Animal makes sound
Dog barks

C:\Users\SLA 1035 Girishma K\Desktop>
```

Types of Inheritance:

1.Single Inheritance:

- One child class inherit from one parent class.

Program:

```
class Animal {  
  
    void sound() {  
  
        System.out.println("Animal makes sound");  
  
    }  
  
}
```

```
public class Dog extends Animal {
```

```
    void bark() {  
  
        System.out.println("Dog barks");  
  
    }
```

```
    public static void main(String[] args) {  
  
        Dog d = new Dog();  
  
        d.sound();  
  
        d.bark();  
  
    }  
}
```

Output:

Animal makes sound

Dog barks

```
C:\Users\SLA 1035 Girishma K\Documents>javac Dog.java

C:\Users\SLA 1035 Girishma K\Documents>java Dog
Animal makes sound
Dog barks

C:\Users\SLA 1035 Girishma K\Documents>
```

2.Multilevel Inheritance:

- A class inherit from another class, and then another class inherit from it.

Program:

```
class Animal {

    void eat() {

        System.out.println("Animal eats");

    }

}
```

```
class Dog extends Animal {

    void bark() {

        System.out.println("Dog barks");

    }

}
```

```
public class Puppy extends Dog {

    void weep() {
```

```
        System.out.println("Puppy weeps");  
    }
```

```
public static void main(String[] args) {  
    Puppy p = new Puppy();  
    p.eat(); // from Animal  
    p.bark(); // from Dog  
    p.weep(); // from Puppy  
}  
}
```

Flowchart:

| Animal |

| eat() |

|

extends

| Dog | | bark() |

extends

|

| Puppy | | weep()

| p = new Puppy(); |

p.eat(); // Animal |

| p.bark(); // Dog |

| p.weep(); // Puppy

```
// Puppy
```

Output:

Animal eats

Dog barks

Puppy Weeps

```
C:\Users\SLA 1035 Girishma K\Documents>javac Puppy.java
C:\Users\SLA 1035 Girishma K\Documents>java Puppy
Animal eats
Dog barks
Puppy weeps
```

3.Hierarchical Inheritance:

- Two or more child classes inherit the properties and methods of the same parent class.

```
class Animal {
    void sound() {
        System.out.println("Animal makes sound");
    }
}
```

```
class Dog extends Animal {
    void bark() {
        System.out.println("Dog barks");
    }
}
```

```

    }
}

public class Cat extends Animal {

    void meow() {

        System.out.println("Cat meows");

    }

    public static void main(String[] args) {

        Dog d = new Dog();

        d.sound();

        d.bark();

        Cat c = new Cat();

        c.sound();

        c.meow();

    }

}

```

Output:

Animal makes sound

Dog barks

Animal makes sound

Cat meows

```
C:\Users\SLA 1035 Girishma K\Documents>javac Cat.java
C:\Users\SLA 1035 Girishma K\Documents>java Cat
Animal makes sound
Dog barks
Animal makes sound
Cat meows
```

4.Hybrid Inheritance:

- Hybrid inheritance is a combination of two or more inheritance types. Java supports hybrid inheritance using interfaces.

Program:

```
interface Pet {
    void play();
}
```

```
class Animal {
    void sound() {
        System.out.println("Animal makes sound");
    }
}
```

```
class Dog extends Animal implements Pet {
    public void play() {
        System.out.println("Dog plays");
    }
}
```

```
public class Cat extends Animal {  
  
    void meow() {  
        System.out.println("Cat meows");  
    }  
  
    public static void main(String[] args) {  
        Dog d = new Dog();  
        d.sound(); // Hierarchical part  
        d.play(); // Multiple part  
  
        Cat c = new Cat();  
        c.sound();  
        c.meow();  
    }  
}
```

Animal makes sound

Dog plays

Animal makes sound

Cat meows

```
C:\Users\SLA 1035 Girishma K\Documents>javac Hybrid.java  
C:\Users\SLA 1035 Girishma K\Documents>java Hybrid  
Animal makes sound  
Dog plays  
Animal makes sound  
Cat meows
```

3. Polymorphism:

Polymorphism means one method can perform different actions. It is achieved through method overloading and method overriding.

Types of Polymorphism

Polymorphism is mainly of **2 types**:

1. Compile-Time Polymorphism (Static Polymorphism)

- Decided at compile time which function/method to call.
- Also known as Early Binding.
- Achieved in two ways:
 - Method Overloading – Multiple methods with the same name but different parameters.

Example: add(int, int) and add(double, double) — same name, different parameters.

2. Runtime Polymorphism (Dynamic Polymorphism)

- Decided at runtime which function to call.
- Also known as Late Binding.
- Achieved through:

- Method Overriding – Redefining a parent class's method in a child class using the virtual keyword.

Example: The sound() method in the Animal class is overridden differently in the Cat and Cow classes.

Method Overriding

Program:

```
public class Polymorphism {  
  
    public void display() {  
        System.out.println("This is Parent Class");  
    }  
  
    public static void main(String[] args) {  
        Child obj = new Child();  
        obj.display();  
    }  
}  
  
class Child extends Polymorphism {  
  
    public void display() {  
        System.out.println("This is Child Class");  
    }  
}
```

Output:

This is Child Class

```
C:\Users\SLA 1035 Girishma K\Desktop>javac Polymorphism.java
C:\Users\SLA 1035 Girishma K\Desktop>java Polymorphism.java
This is Child Class
```

Method Overloading

Program:

```
public class Polymorphism {
    public void display() {
        System.out.println("This is Parent Class");
    }

    // Overloaded method - same name, different parameters
    public void display(String name) {
        System.out.println("This is Parent Class, Hello " + name);
    }

    // Overloaded method - different number of parameters
    public void display(String name, int age) {
        System.out.println("This is Parent Class, " + name + " is " + age + " years old");
    }

    public static void main(String[] args) {
        Polymorphism obj = new Polymorphism();

        obj.display();
        obj.display("Kiran");
        obj.display("Kiran", 25);
    }
}
```

```
}  
}
```

```
C:\Users\SLA 1035 Girishma K\Documents>javac Polymorphism.java  
C:\Users\SLA 1035 Girishma K\Documents>java Polymorphism  
This is Parent Class  
This is Parent Class, Hello Kiran  
This is Parent Class, Kiran is 25 years old  
C:\Users\SLA 1035 Girishma K\Documents>
```

4. Abstraction:

Abstraction hides implementation details and shows only important information to the user.

Program:

```
public abstract class Abstraction {  
  
    abstract void display();  
  
    public static void main(String[] args) {  
        Demo obj = new Demo();  
        obj.display();  
    }  
}  
  
class Demo extends Abstraction {  
  
    void display() {  
        System.out.println("Abstract Method Implemented");  
    }  
}
```

Output:

Abstract Method Implemented

```
C:\Users\SLA 1035 Girishma K\Desktop>javac Abstraction.java
C:\Users\SLA 1035 Girishma K\Desktop>java Abstraction.java
Abstract Method Implemented
```